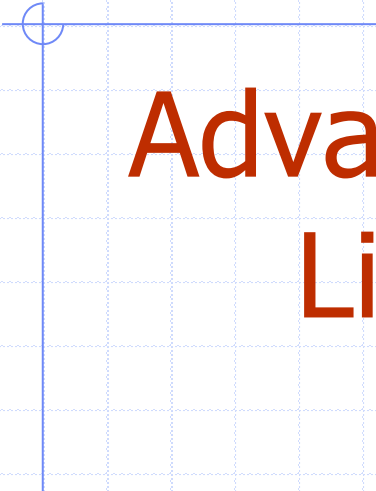




Week 3

Advanced Sorting Algorithms

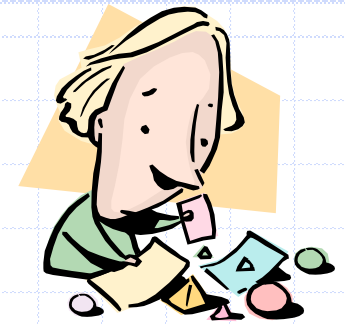
Linear Data Structures

Amortization

Algorithms and Data Structures
COMP3506/7505

Week 3 – Sorting & Linear DS

1. Bucket-sort and radix-sort
2. Arrays
3. Linked Lists
4. Extensible Lists and Amortization



Comparison Sorts

- Sorted order determined by comparing keys

```
if  $A[k] < A[k+1]$ : —  
    # Do some swapping
```

- Examples
 - Bubble, insertion, selection, merge and quick sorts
 - ◆ Everything we've seen so far!

“Hang on, last week you said”

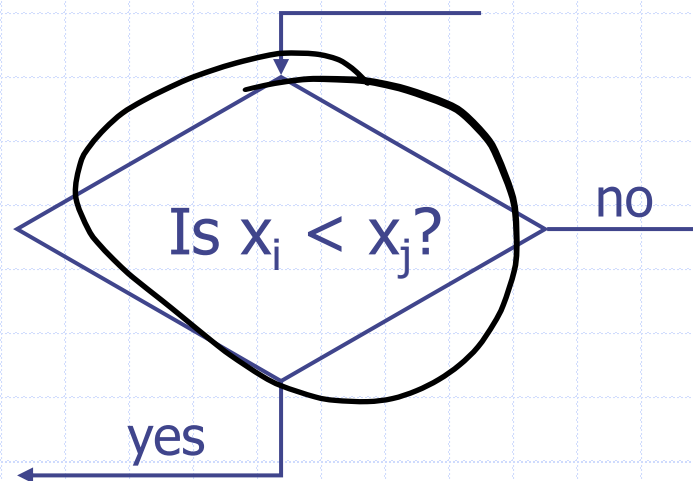
- I told you that it doesn't get any better than *mergesort*
 - That is, $O(n \log_2 n)$ worst case complexity
 - In fact, it is $\Omega(n \log_2 n)$...
 - So, it is $\Theta(n \log_2 n)$
- Do you believe me? 😊



Determining Performance

Ω

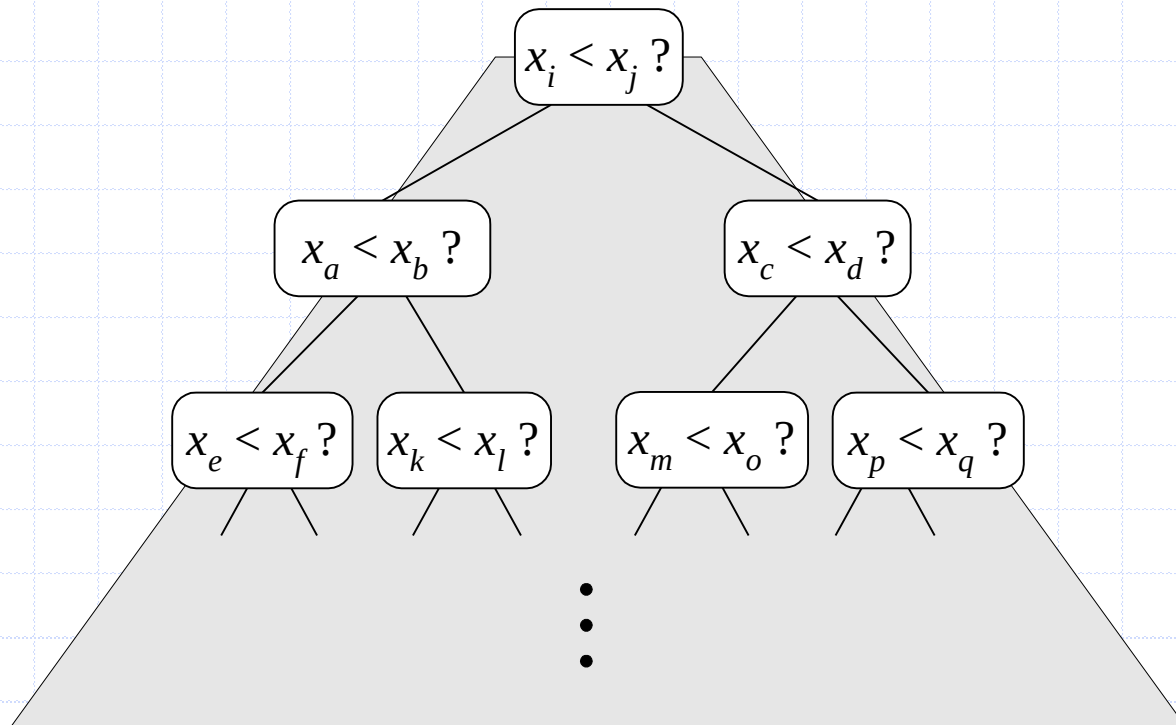
- Derive a worst-case lower bound on running time of any algorithm that uses comparisons to sort n elements, $x_1, x_2, \dots, x_n$



$n \log n$

Counting Comparisons

- Each possible run of the algorithm corresponds to a root-to-leaf path in a **decision tree**



Think Permutations...

Q: How many permutations of a list L of length $|L|$?

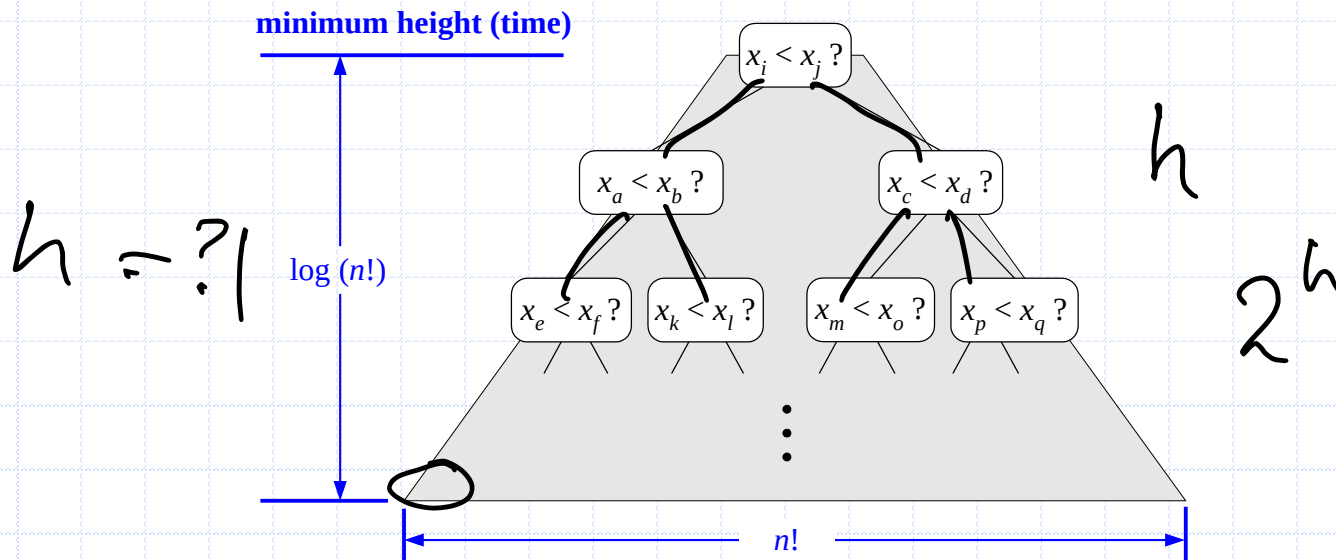
$\langle A, B, C \rangle \quad |L| = 3$

$$3! = 3 \cdot 2 \cdot 1 = 6$$

-	ABC		6
-	ACB		
-	BAC		
-	BCA		
-	CBA		
-	CAB		

Decision Tree Height

- Height of decision tree is a **lower bound** on running time
- Every unique input permutation must lead to a separate leaf output
 - if not, some input ...4...5... would have same output ordering as ...5...4..., which would be wrong
- There are $n! = 1 \cdot 2 \cdot \dots \cdot n$ leaves – height is at least **$\log(n!)$**



Proof.

Idea: To get to bottom of decision tree,
we do at most h comparisons.
This tree has at most 2^h leaves.
Has to be at least $n!$ leaves...

So what is h ? ..

$$2^h \geq n! \Rightarrow h \geq \log(n!)$$

$$= \log(n(n-1)(n-2) \cdots (2))$$

$$= \underbrace{\log n}_n + \underbrace{\log(n-1)}_{n-1} + \underbrace{\log(n-2)}_{n-2} + \cdots + \log 2$$

$$= \sum_{i=2}^n \log i \quad \checkmark$$

$$= \underbrace{\sum_{i=2}^{n/2-1} \log i}_0 + \underbrace{\sum_{i=n/2}^n \log i}$$

$$\geq \underbrace{0}_{\geq 0} + \sum_{i=n/2}^n \log \frac{n}{2}$$

$$= \frac{n}{2} \cdot \log \frac{n}{2}$$

$$= \Omega(n \log n) \quad \checkmark \quad \checkmark$$

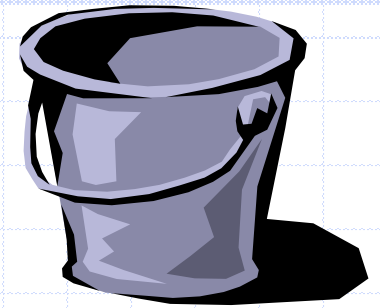
Lower Bound

- Comparison-based sorting algorithms takes at least $\log(n!)$ time
- $\therefore$ any such algorithm takes at least
 - $\log(n!) \geq \log \left(\frac{n}{2}\right)^{n/2} = (n/2) \log(n/2)$
- Any comparison-based sorting algorithm must run in $\Omega(n \log n)$ time
 - merge and heap sorts are asymptotically optimal
 - ◆ no other comparison sorts are faster by more than a constant factor

Can We Do Better?

- Do we have to compare every key to sort data?

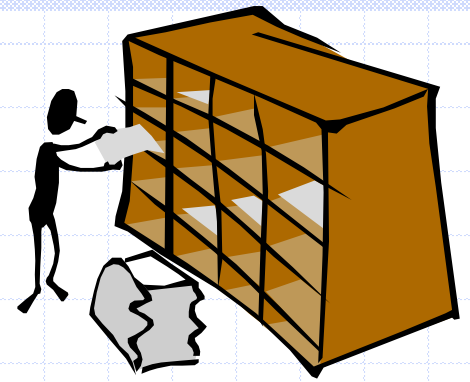




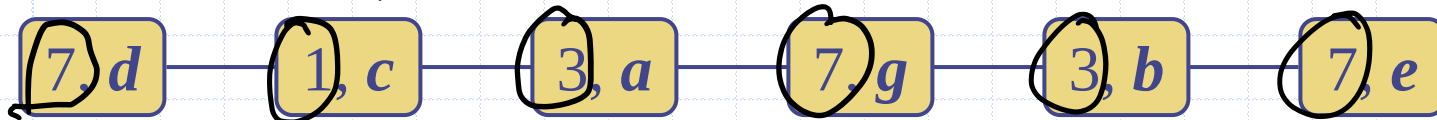
Bucket-Sort

- Let S be a list of n (key, element) items with keys in the range $[0, N - 1]$
- Bucket-sort uses the keys as indices into an auxiliary array B of lists (buckets)
 - **Phase 1:** Empty list S by moving each entry (k, o) into its bucket $B[k]$
 - **Phase 2:** For $i = 0, \dots, N - 1$, move the entries of bucket $B[i]$ to the end of list S

Example



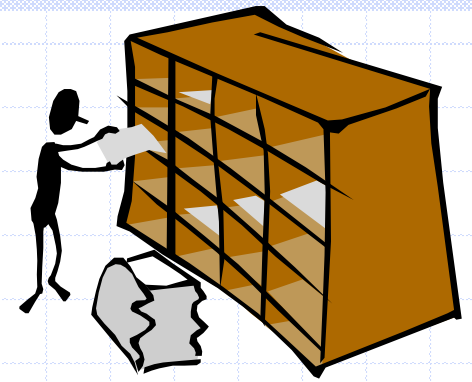
□ Key range $[0, 9]$



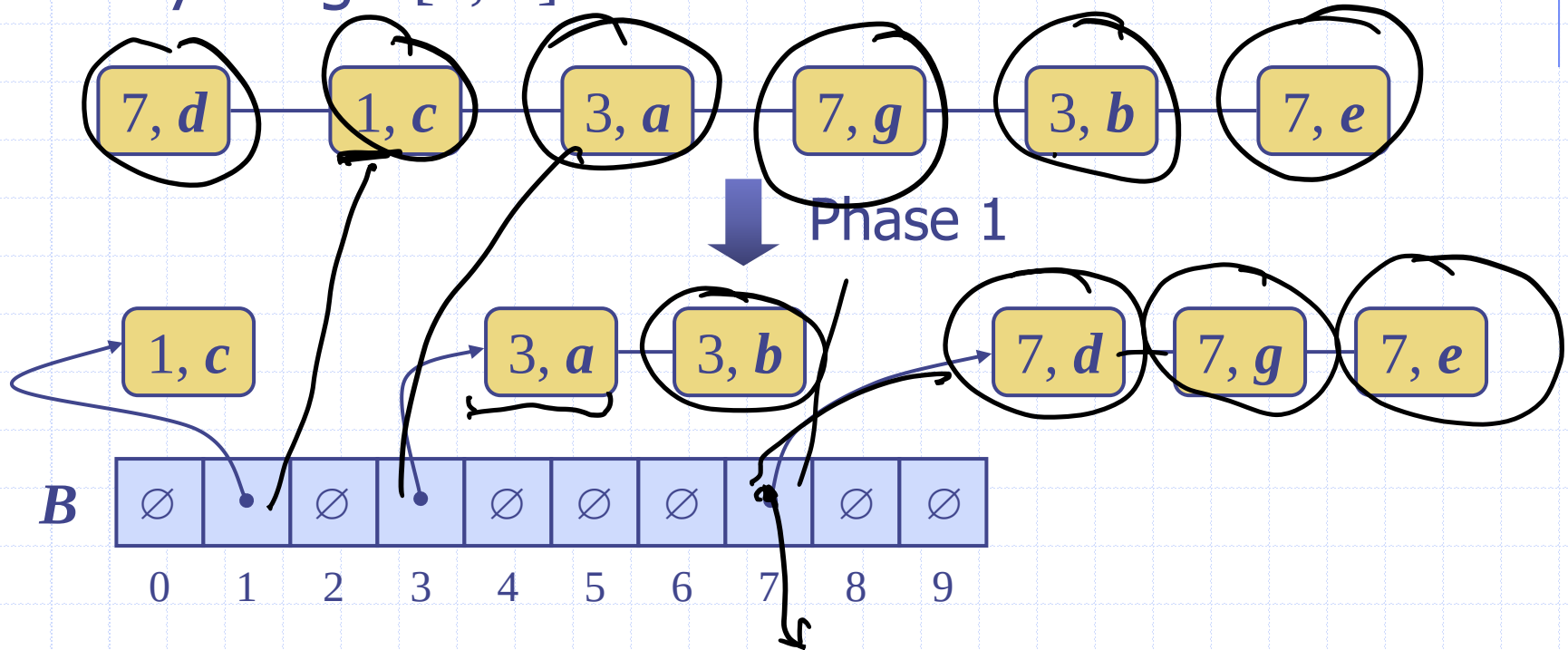
B

$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
0	1	2	3	4	5	6	7	8	9

Example

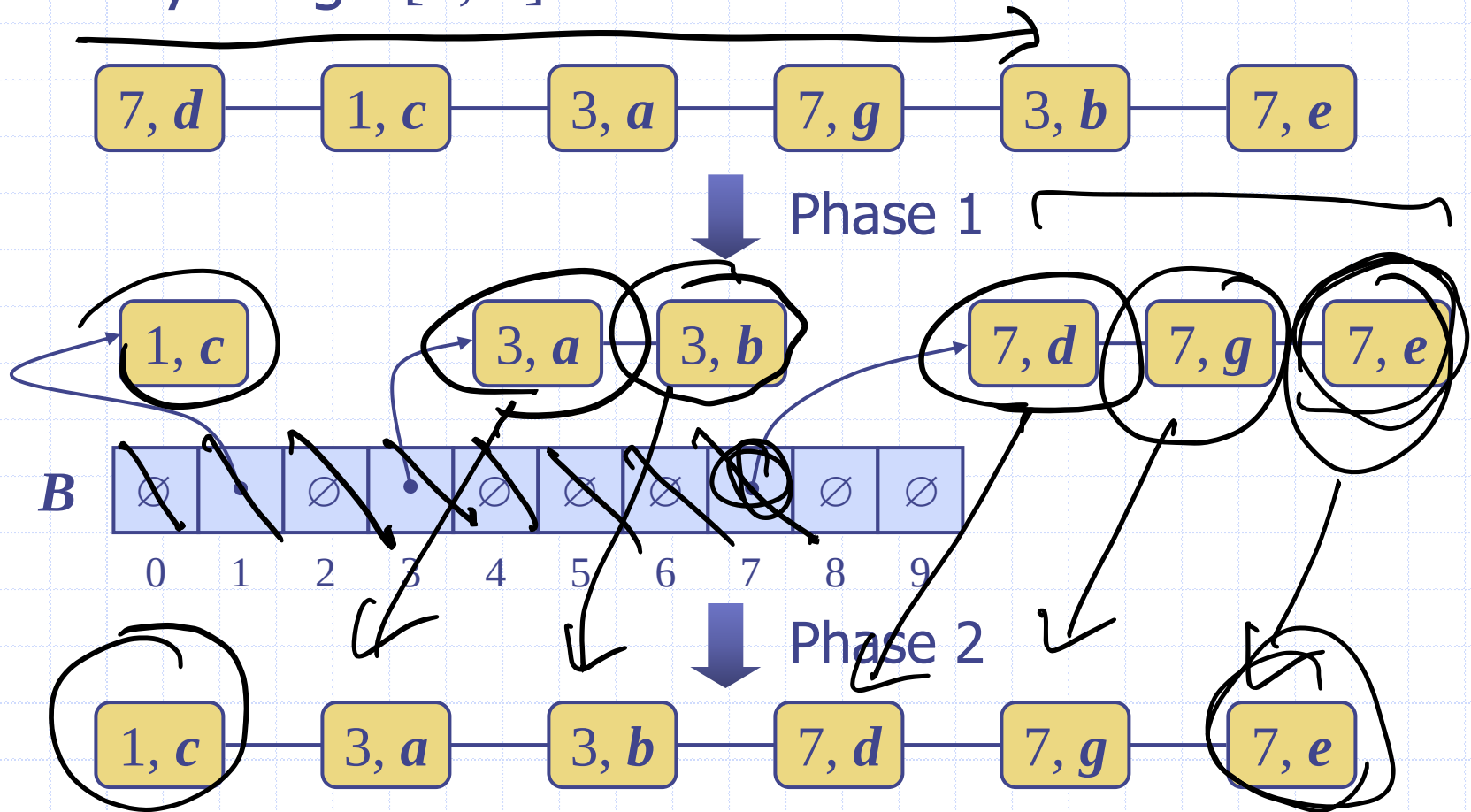


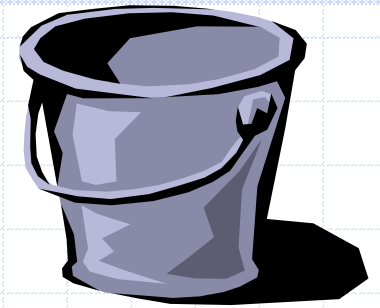
□ Key range $[0, 9]$



A cartoon illustration of a person in a black silhouette placing a white envelope into a brown mailbox. The mailbox is a large, rectangular structure with a grid of compartments. A white envelope is shown falling from the person's hand into one of the compartments. The background is a light blue grid pattern.

- 1





Bucket-Sort

Algorithm *bucketSort*(S):

Input: sequence S of n entries with integer keys in the range $[0, N - 1]$

Output: sequence S sorted in nondecreasing order of the keys

$B \leftarrow$ array of N empty sequences

for each entry e **in** S **do**

$k \leftarrow$ key of e

 remove e from S

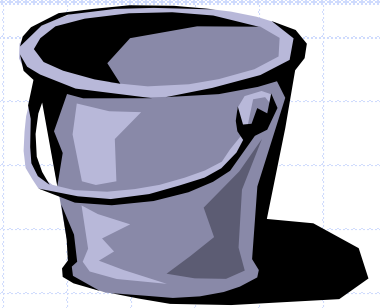
 insert e at the end of bucket $B[k]$

for $i \leftarrow 0$ **to** $N-1$ **do**

for each entry e **in** $B[i]$ **do**

 remove e from $B[i]$

 insert e at the end of S



Bucket-Sort

Algorithm *bucketSort*(S):

Input: sequence S of n entries with integer keys in the range $[0, N - 1]$

Output: sequence S sorted in nondecreasing order of the keys

$B \leftarrow$ array of N empty sequences

for each entry e **in** S **do**

$k \leftarrow$ key of e

remove e from S

insert e at the end of bucket $B[k]$

for $i \leftarrow 0$ **to** $N-1$ **do**

for each entry e **in** $B[i]$ **do**

remove e from $B[i]$

insert e at the end of S

□ Analysis

- Initialising the bucket array takes $O(N)$ time

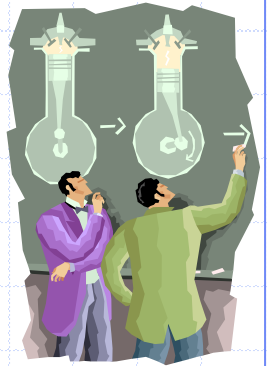
- Phase 1 takes $O(n)$ time

- Phase 2 takes $O(n + N)$ time

□ Bucket-Sort takes $O(n + N)$ time

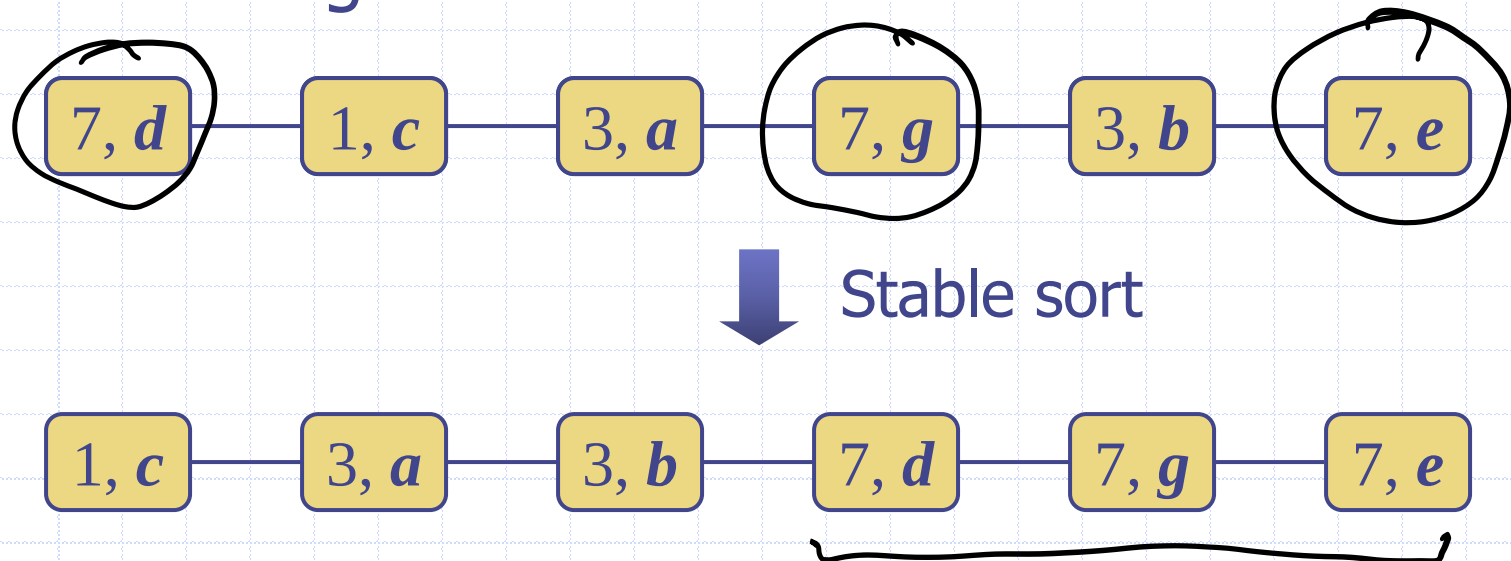
$O(n)$
 $O(N)$

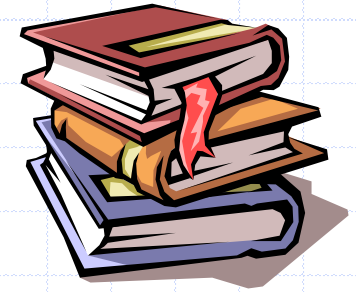
Properties and Extensions



□ Stable Sort Property

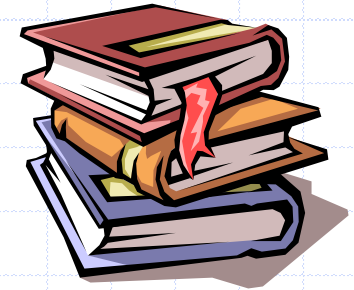
- **Relative order** of any two items with the same key is **preserved** after the execution of the algorithm





Lexicographic Order

- d -tuple is a sequence of d keys $(k_1, k_2, \dots, k_d)$, where key k_i is said to be the i -th dimension of the tuple
 - e.g. Cartesian coordinates of a point in space are a 3-tuple
 - e.g. ant < apple
 η ρ



Lexicographic Order

- Lexicographic order of two d -tuples is recursively defined as follows

$$(x_1, x_2, \dots, x_d) < (y_1, y_2, \dots, y_d)$$



$$x_1 < y_1 \vee x_1 = y_1 \wedge (x_2, \dots, x_d) < (y_2, \dots, y_d)$$

- i.e. tuples are compared by the first dimension, then by the second dimension, etc.

- $(2, 1, 4) < (3, 2, 5)$

since $\underline{2} < \underline{3}$

- $(2, 1, 4) < (2, 2, 5)$

since $\underline{2} = \underline{2}$ and $\underline{1} < \underline{2}$

- $(2, 1, 4) < (2, 1, 5)$

since $2 = 2, 1 = 1$ and $4 < 5$

Handwritten example with arrows pointing to dimensions: $(2, 1, 4) < (3, 2, 5)$

- 246 214 324 515 746
 214 515 324 246 746
 - 214 324 515 246 746 ✓

not sorted
 ops

Lexicographic-Sort (aka Tuple Sort)

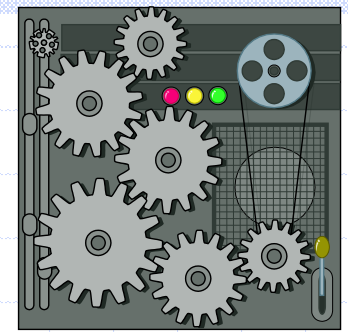
- ◆ C_i – comparator that compares two tuples by their i -th dimension
- ◆ $stableSort(S, C)$ – stable sorting algorithm that uses comparator C
- ◆ $lexicographicSort$ sorts a sequence of d -tuples in lexicographic order by executing $stableSort$, d times
 - once per dimension
- ◆ Runs in $O(d \cdot T(n))$ time
 - where $T(n)$ is the running time of $stableSort$

Algorithm *lexicographicSort*(S)
Input sequence S of d -tuples
Output sequence S sorted in lexicographic order

for $i \leftarrow d$ down to 1 * careful here!
 $stableSort(S, C_i)$

Example

↓
 (7,4,6) (5,1,5) (2,4,6) (2,1,4) (3,2,4)
 (2,1,4) (3,2,4) (5,1,5) (7,4,6) (2,4,6)
 (2,1,4) (5,1,5) (3,2,4) (7,4,6) (2,4,6)
 (2,1,4) (2,4,6) (3,2,4) (5,1,5) (7,4,6)



Radix-Sort

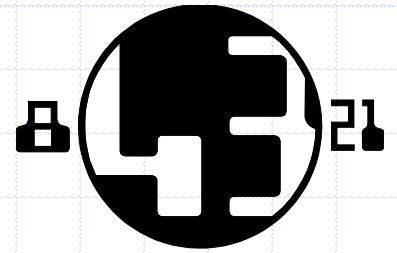
- Specialisation of lexicographic-sort
 - uses bucket-sort as the stable sorting algorithm in each dimension
- Applicable to tuples where the keys in each dimension i are integers in the range $[0, N - 1]$
- Runs in time $O(d \cdot (n + N))$

Algorithm *radixSort*(S, N)

Input sequence S of d -tuples such that $(0, \dots, 0) \leq (x_1, \dots, x_d)$ and $(x_1, \dots, x_d) \leq (N - 1, \dots, N - 1)$ for each tuple $(x_1, \dots, x_d)$ in S

Output sequence S sorted in lexicographic order

for $i \leftarrow d$ **downto** 1
 bucketSort(S, N)



Radix-Sort for Binary Numbers

- Consider a sequence of n b -bit integers
 - $x = x_{b-1} \dots x_1 x_0$ / (00101101)
- Represent each element as a b -tuple of integers in the range $[0, 1]$ and apply radix-sort with $N = 2$
- Runs in $b \cdot (n+2)$ or $O(b \cdot n)$ time

Algorithm *binaryRadixSort(S)*

Input sequence S of b -bit integers

Output sequence S sorted

replace each element x of S with item $(0, x)$

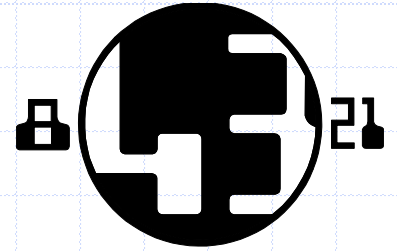
for $i \leftarrow 0$ **to** $b - 1$

replace key k of item (k, x) with bit x_i of x

bucketSort(S, 2)

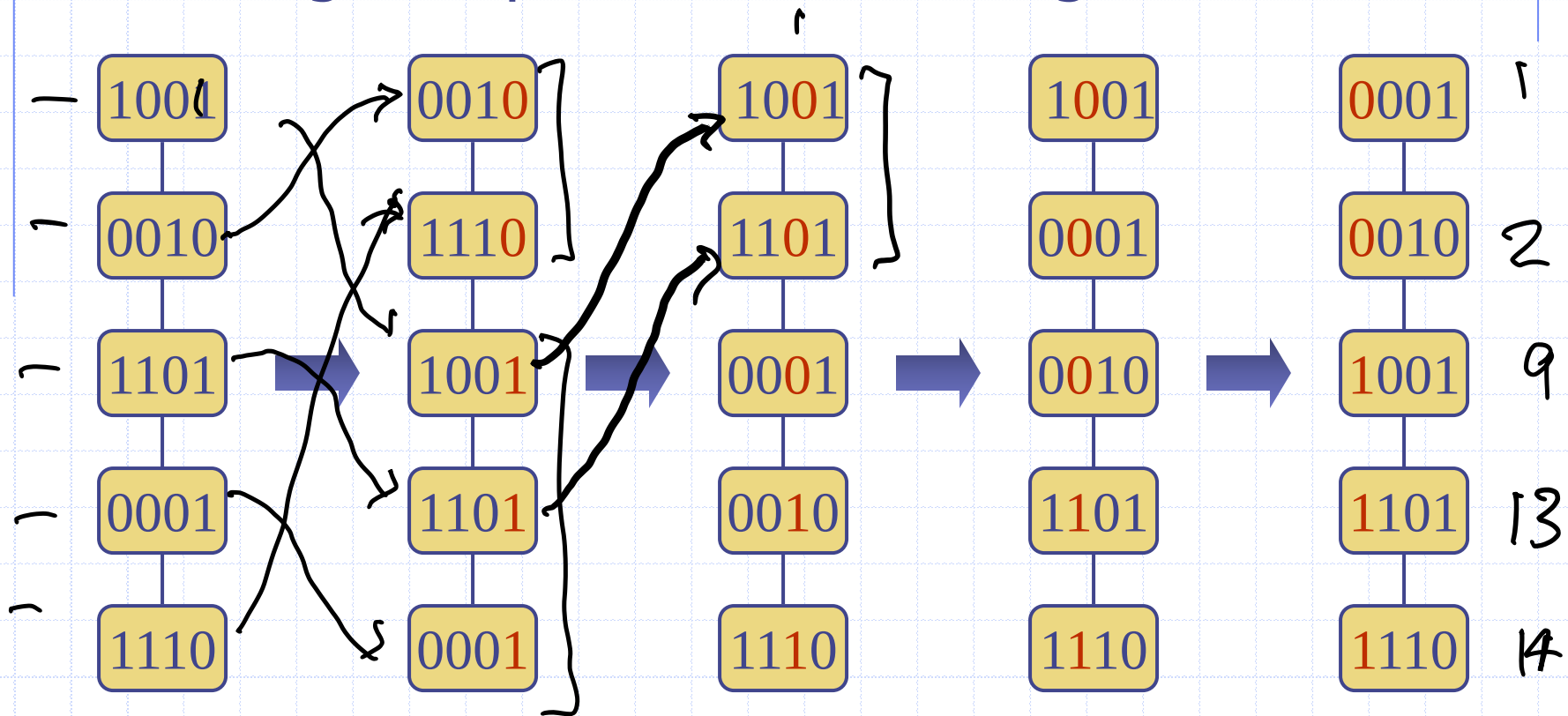
32 33

1 0 0 0 0 0
1 0 0 0 0 1
↑ ↑ ↑ ↑ ↑ ↑

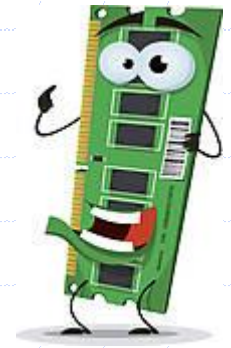


Example

- Sorting a sequence of 4-bit integers



Memory Usage



- Original sequence and bucket array

- $O(n + \textcircled{N})$

- Sort: 10, 999, 3, 100 000 000, 20

◆ Bucket Sort

- $O(5 + 100\,000\,000)$

◆ Radix Sort

- $O(5 + 2)$

Week 3 – Sorting & Linear DS

1. Bucket-sort and radix-sort
2. Arrays
3. Linked Lists
4. Extensible Lists and Amortization

General Linear Structures

A linear structure is one whose elements can be seen as being in a sequence. That is, one element follows the next.

- Lists ✓
 - Stacks ✓
 - Queues ✓
 - Vectors ✓
- Recall the static sequence from Lec 1

Static Sequence ADT

□ Given a list of items X in some order: $x_1, x_2, \dots, x_n$

`build(X) :` Make new data structure for items in X

`len(X) :` Return n

`get(i) :` Return the element at position i

`set(i, x) :` Set x_i to x

□ Note that the way we store the data and compute those functions depend on the *data structure we use*

Dynamic Sequence ADT

- Given a list of items X in some order: $x_1, x_2, \dots, \underline{x_n}$

`build(X)` : Make new data structure for items in X

`len(X)` : Return $n + 1$

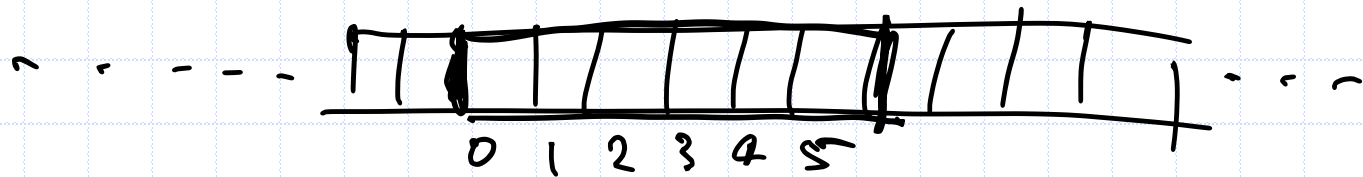
`get(i)` : Return the element at position i

`set(i, x)` : Set x_i to x

`add(x)` : Add x to X

Implementing Linear Structures

- Arrays: Sequence of consecutive memory cells
- Size must be specified at creation (static!)
- What does it get us?
 - Constant time random access – nice!
 - But what if we want to insert something?
 - What if we need more space?



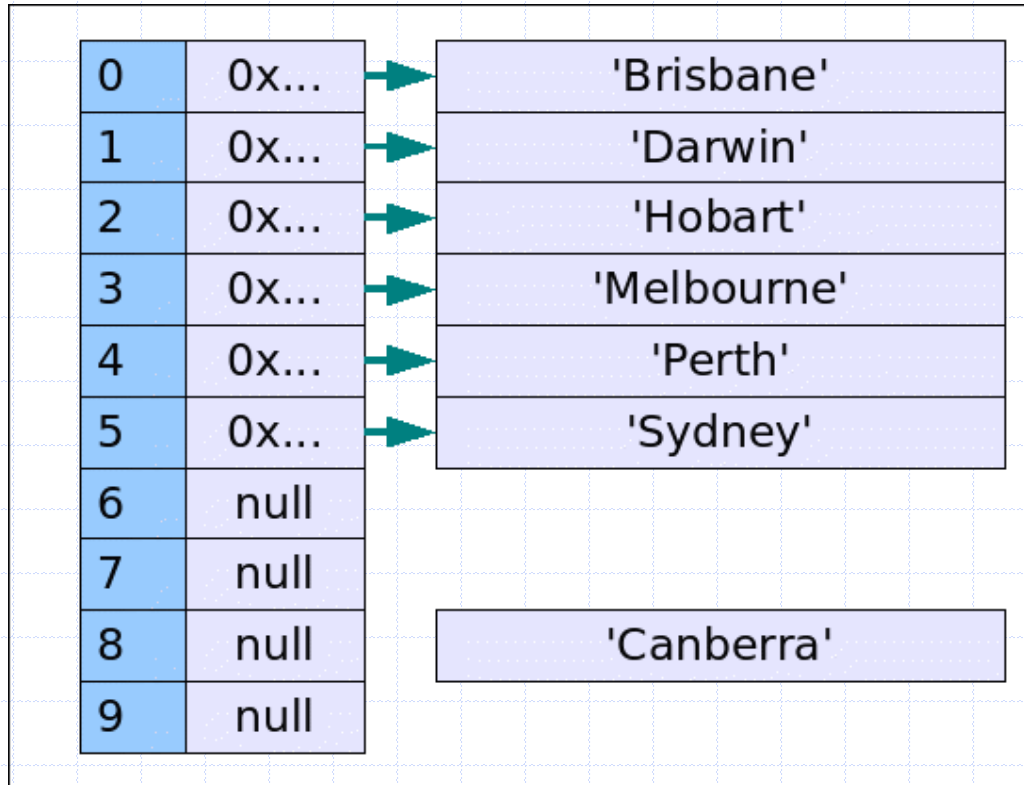
Arrays (insert)

Insert the value 'Canberra', so that the array maintains sorted order

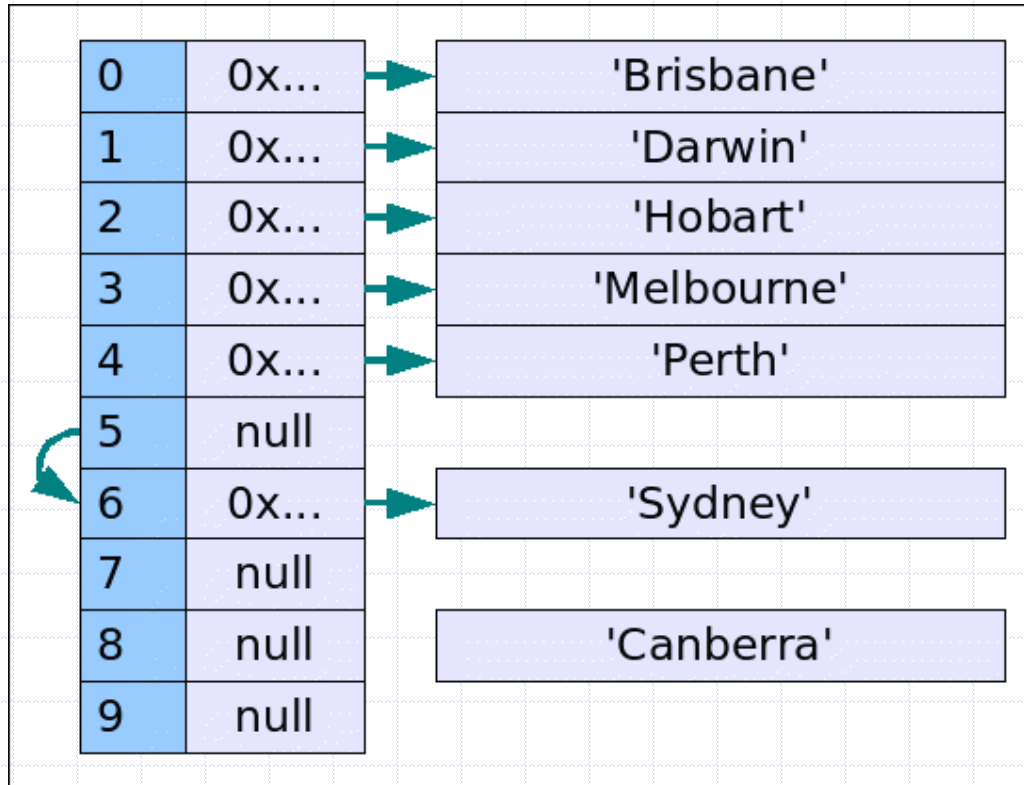
10

0	0x...	→	'Brisbane'
1	0x...	→	'Darwin'
2	0x...	→	'Hobart'
3	0x...	→	'Melbourne'
4	0x...	→	'Perth'
5	0x...	→	'Sydney'
6	null		
7	null		
8	null		
9	null		

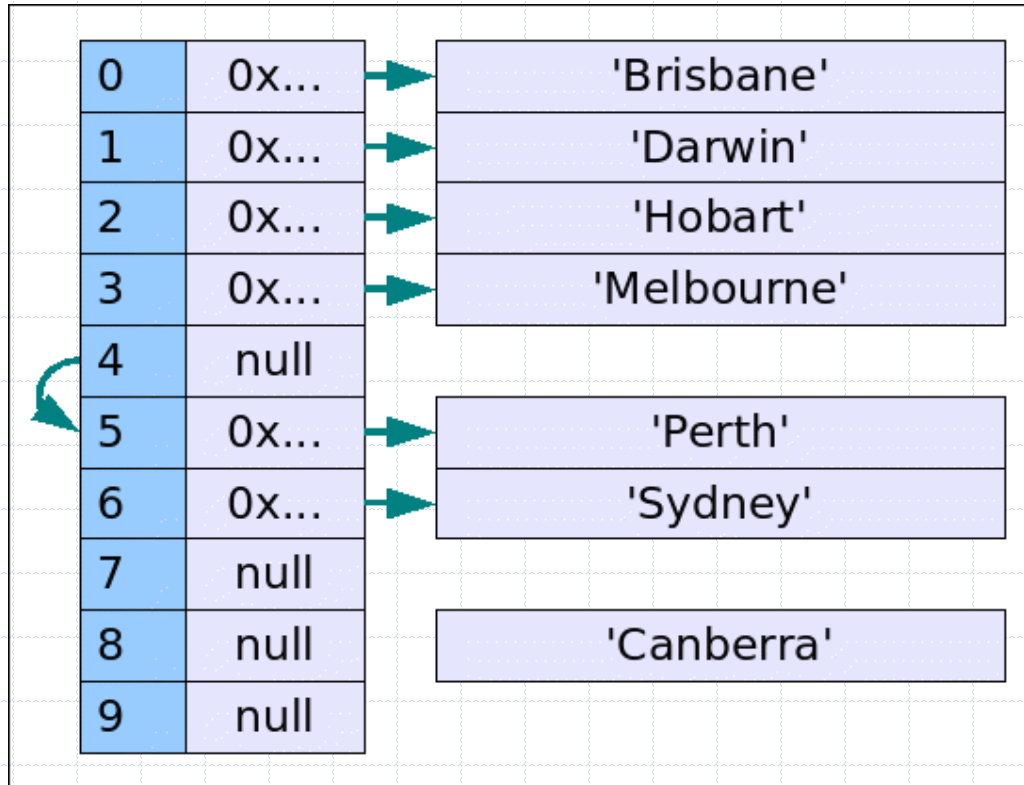
Arrays (insert)



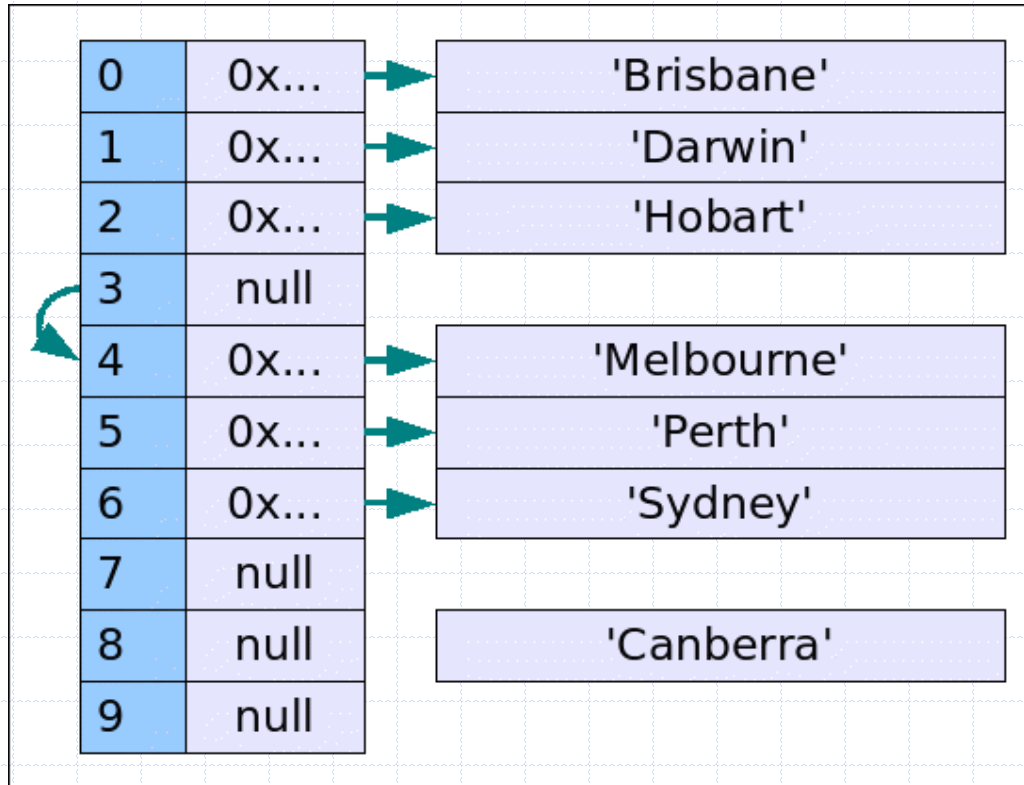
Arrays (insert)



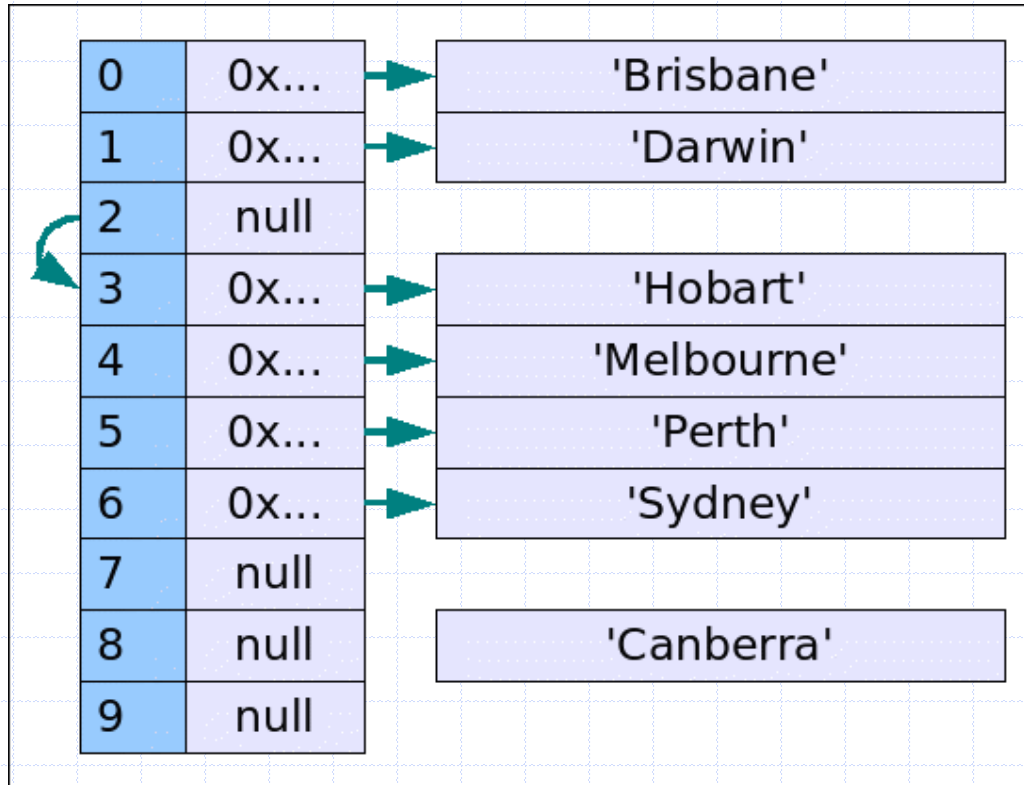
Arrays (insert)



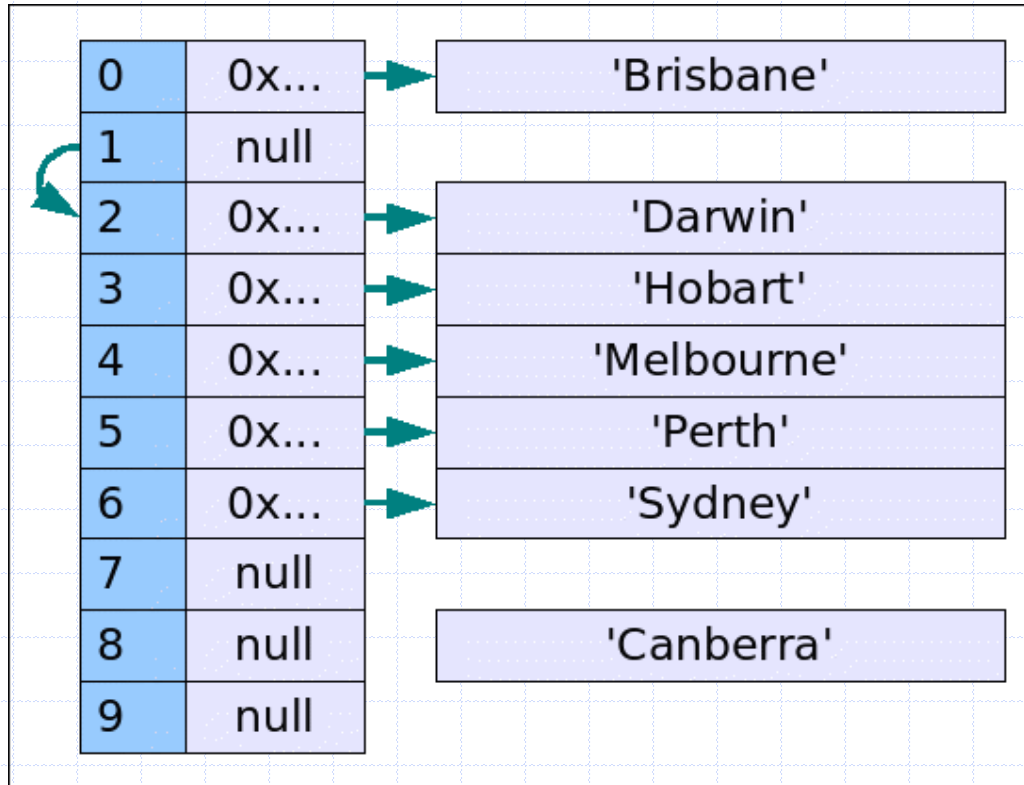
Arrays (insert)



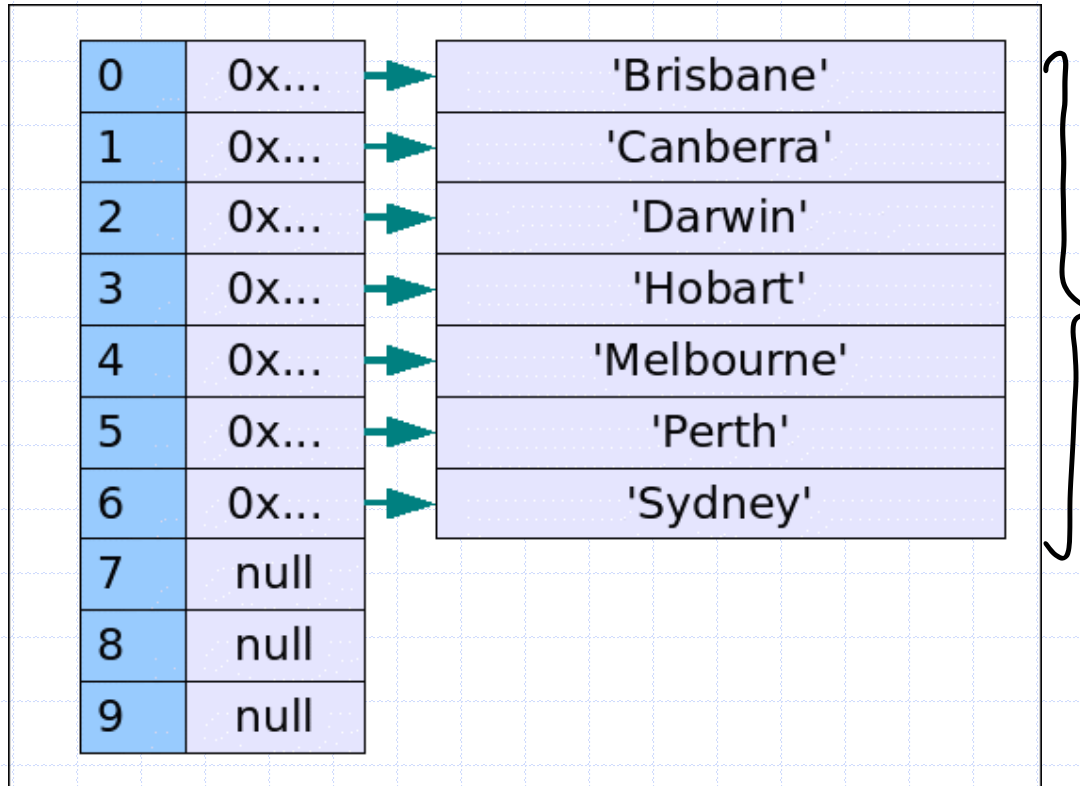
Arrays (insert)



Arrays (insert)



Arrays (insert)



Array Implementation Efficiency

- Accessing an element (by index)

- $O(1)$

- Iterating over elements

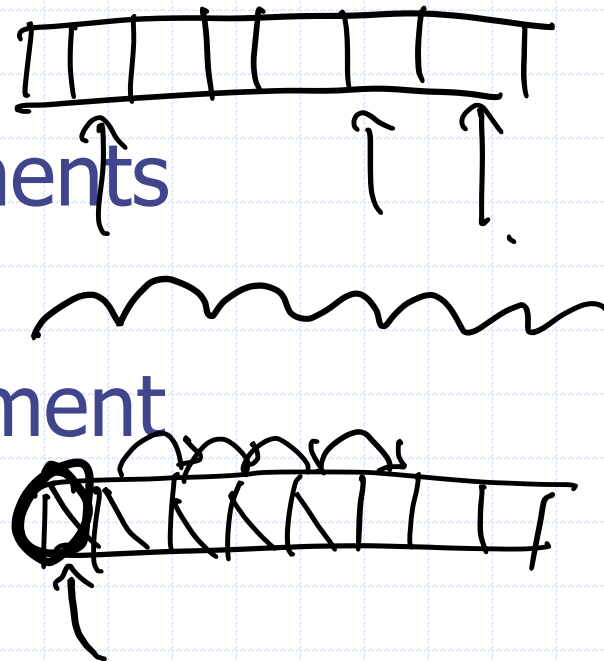
- $O(n)$

- Insert / Delete element

- $O(n)$ ✓

- Memory usage

- $O(n)$

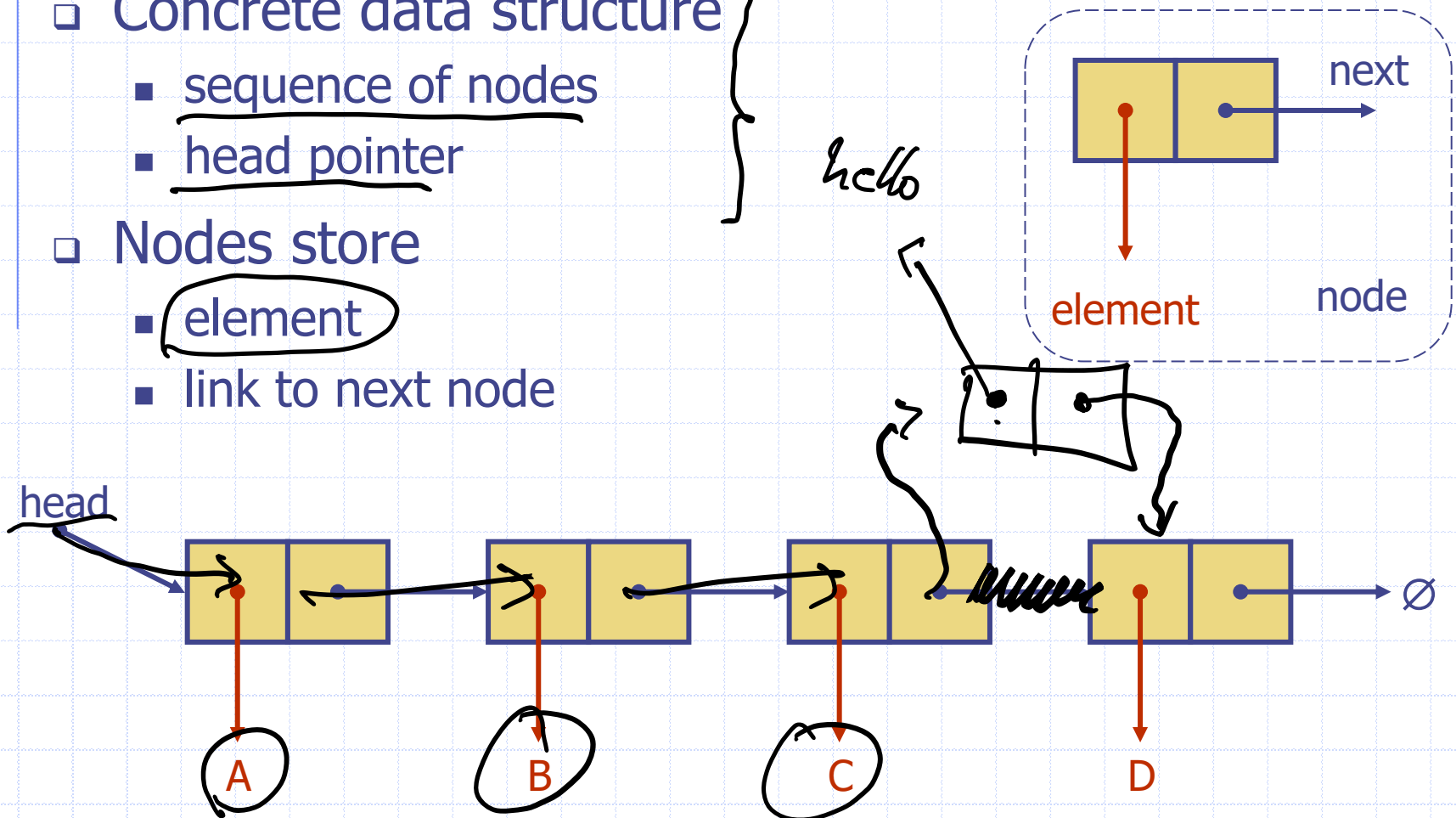


Week 3 – Sorting & Linear DS

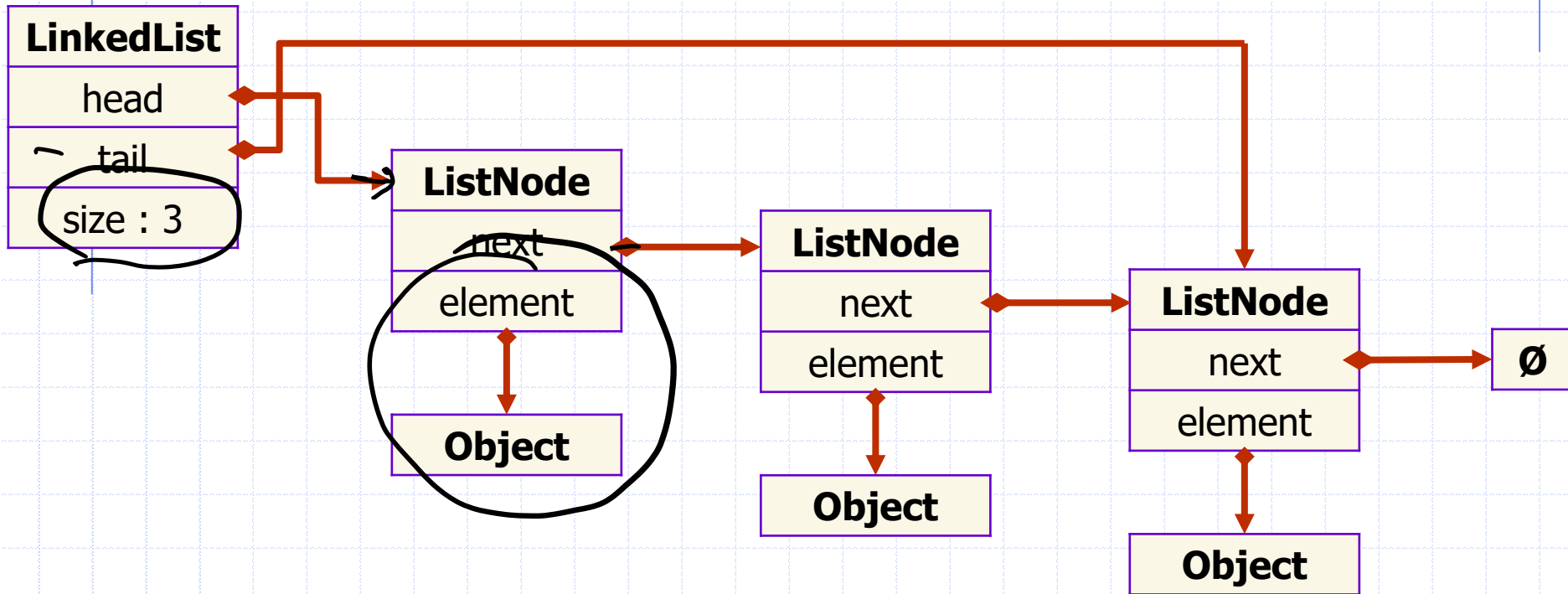
1. Bucket-sort and radix-sort
2. Arrays
3. Linked Lists
4. Extensible Lists and Amortization

Singly Linked List

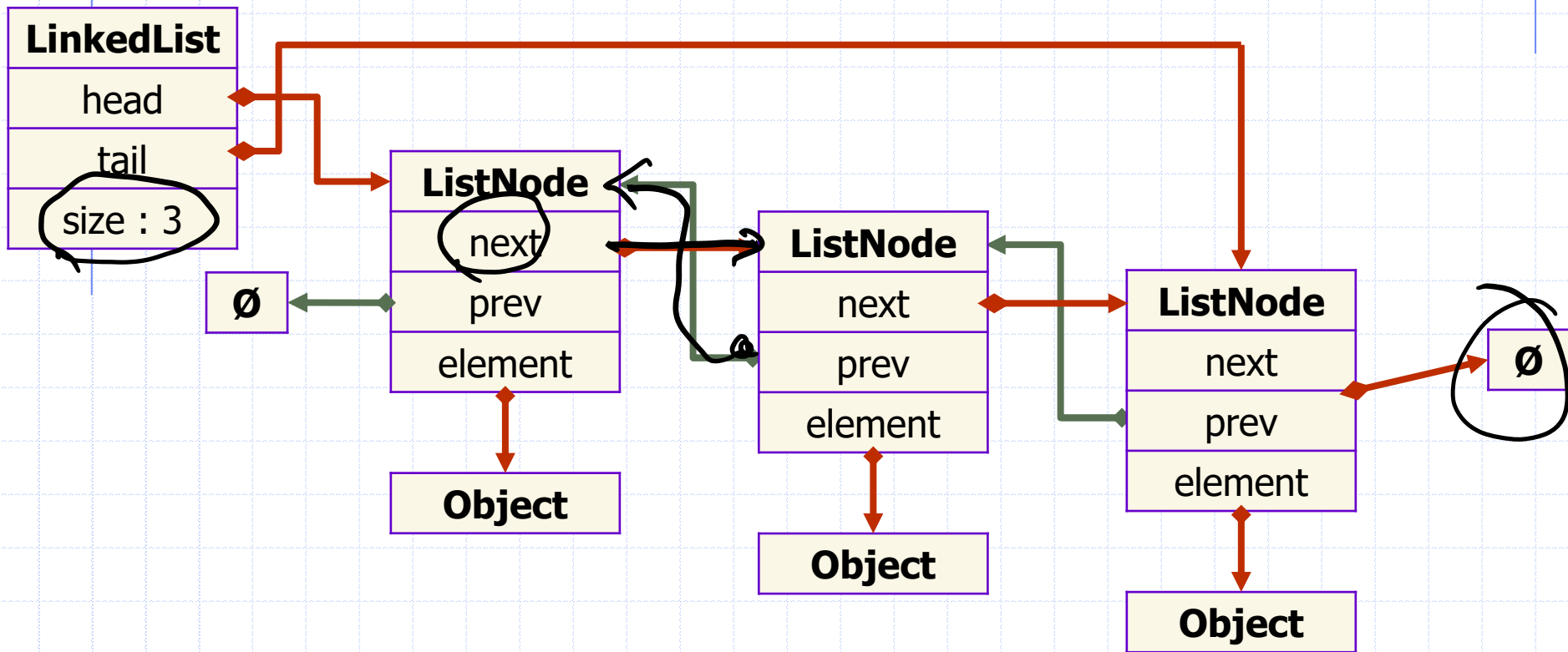
- Concrete data structure
 - sequence of nodes
 - head pointer
- Nodes store
 - element
 - link to next node



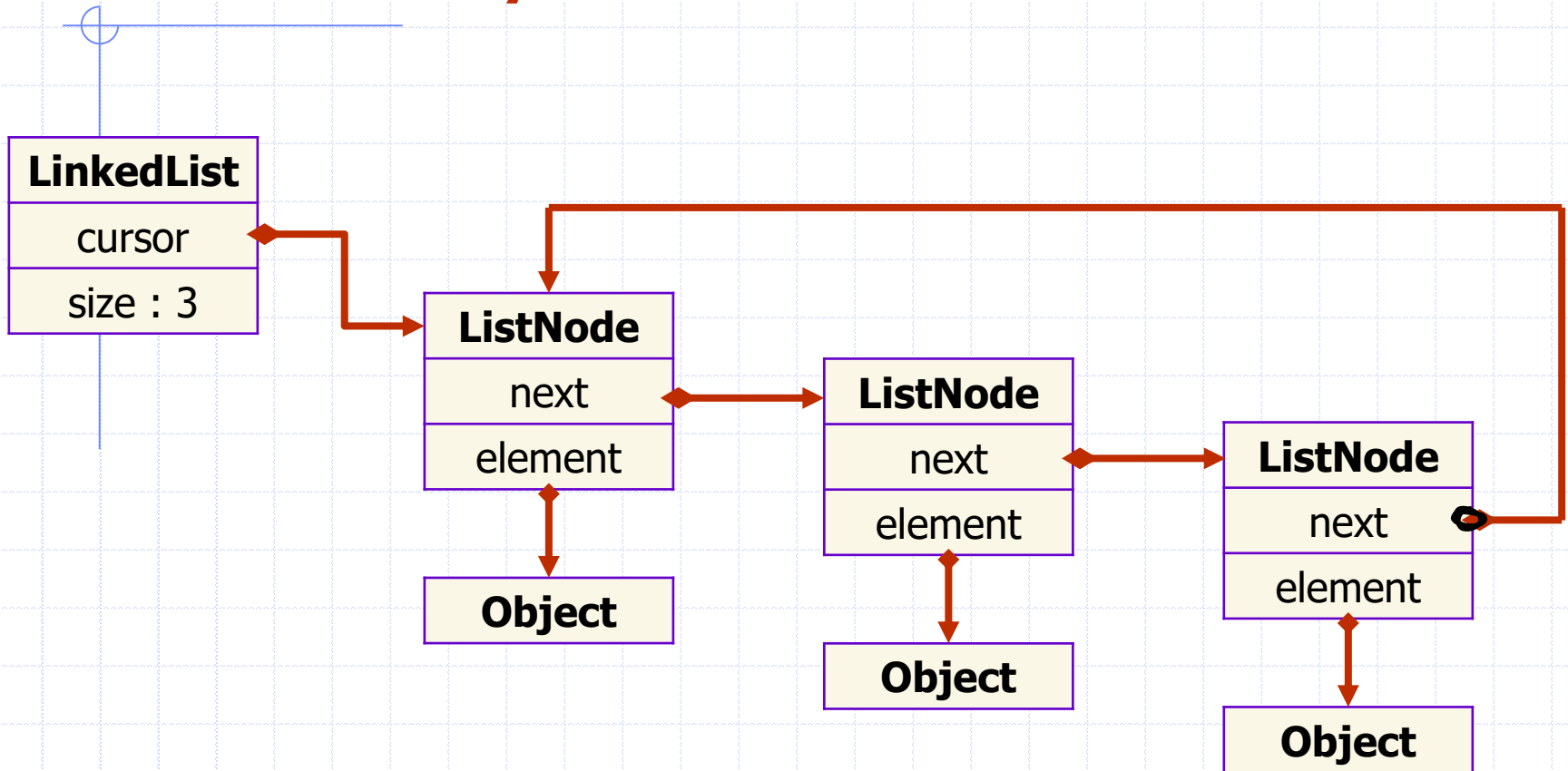
Singly Linked List



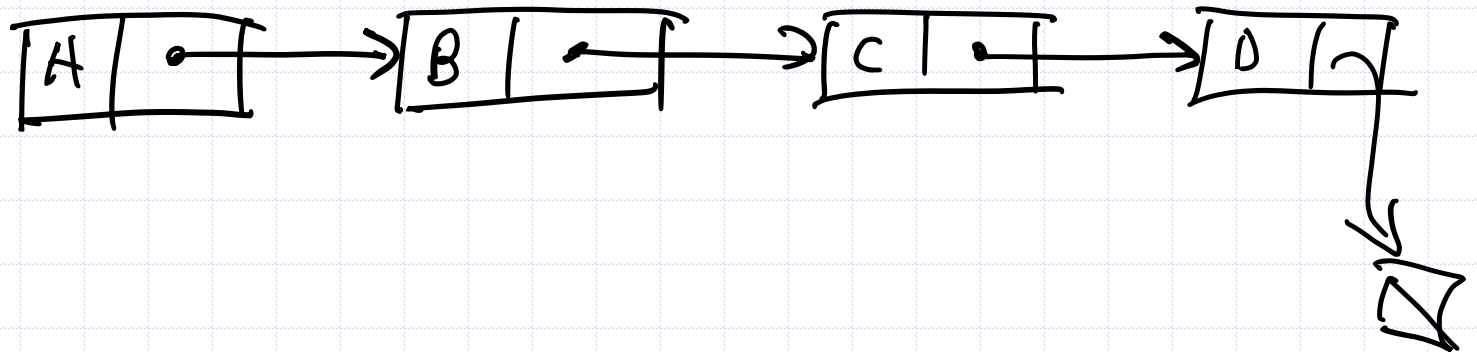
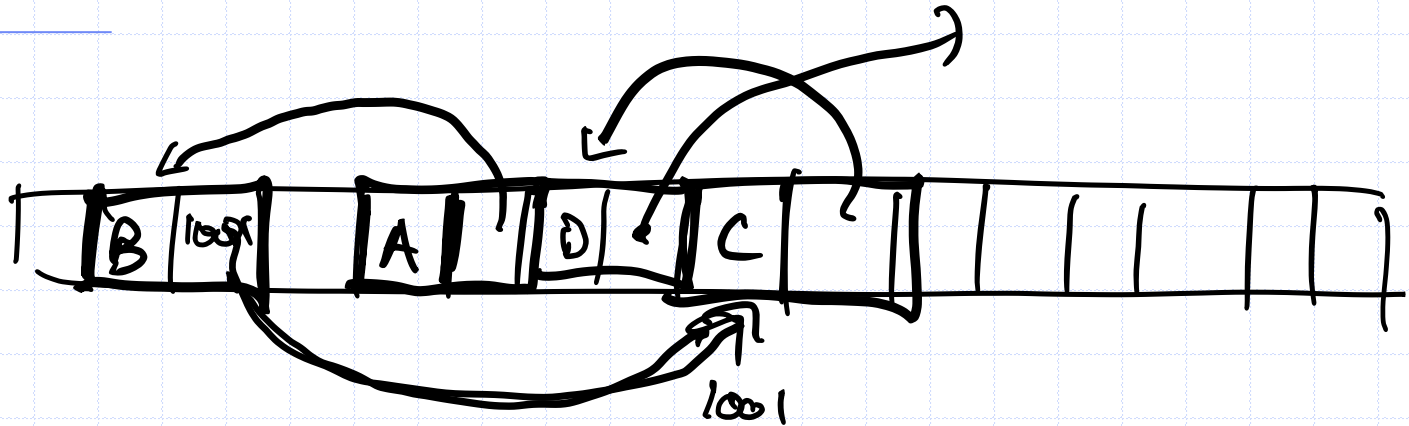
Doubly Linked List



Circularly Linked List



Linked List Sketch



Joel's note to self: We will draw the in-memory layout of a linked list (word RAM)

ASSIGNMENT 1



A python linked list

```
class Node:
    def __init__(self, data):
        self.__data = data
        self.__next = None
```

```
def set_data(self, data):
    self.__data = data
```

...

```
def get_next(self):
    return self.__next
```

```
class LinkedList:
```

```
    def __init__(self):
```

```
        self.__head = None
```

```
        self.__size = 0
```

...

```
def insert_to_front(self, node):
```

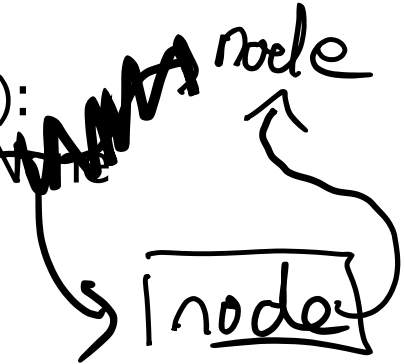
```
    if self.__head != None:
```

```
        node.set_next(self.get_head())
```

```
        self.__head = Node
```

```
        self.size += 1
```

...



Linked List Implementation Efficiency

- Accessing head

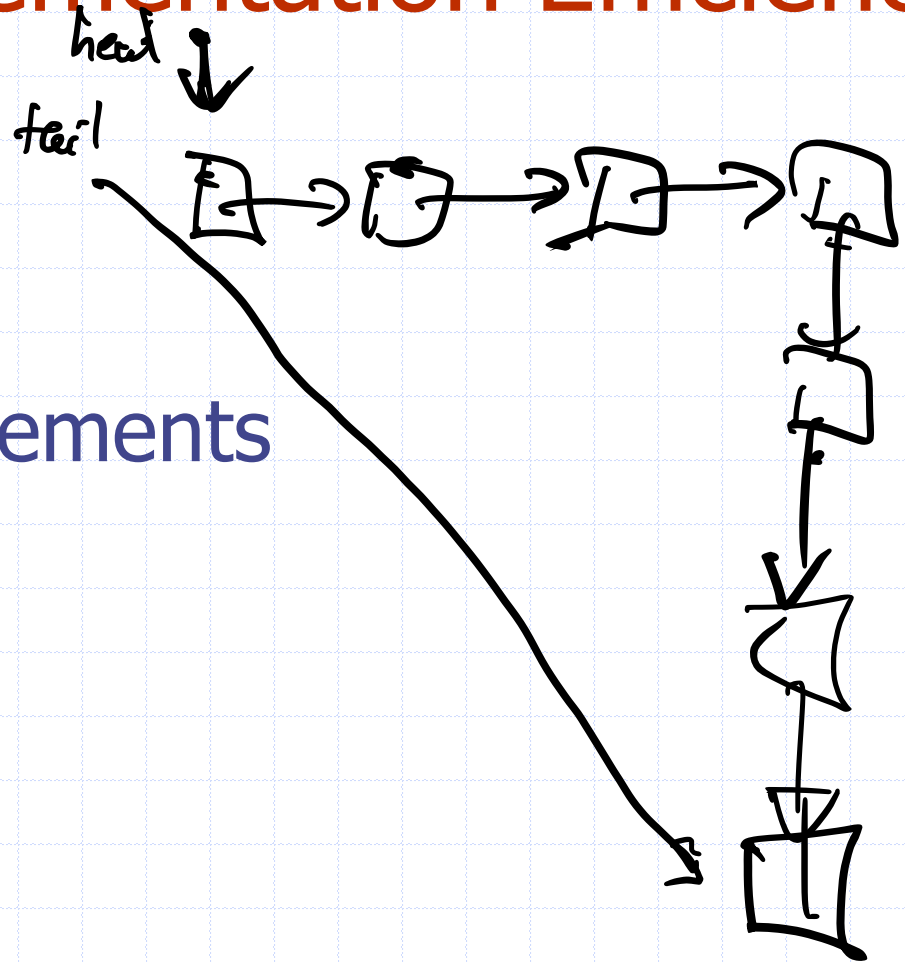
- $O(1)$ ✓

- Iterating over elements

- $O(n)$ ✓

- Memory usage

- $O(n)$ ✓



Note: Augmentation

□ Accessing tail

- $O(n)$ ☹️

- How can we do better? Easy! $O(1)$

```
class LinkedList:  
    def __init__(self):  
        self.__head = None  
        self.__tail = None  
        self.__size = 0
```

```
...
```

Week 3 – Sorting & Linear DS

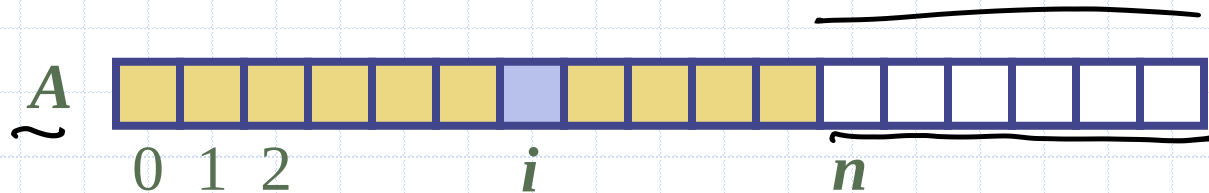
1. Bucket-sort and radix-sort
2. Arrays
3. Linked Lists
4. Extensible Lists and Amortization
5. Comparing Linear Structures

Example: Sequence of List operations

Method	Return Value	List Contents
<u>add(0, A)</u>	—	(A)
<u>add(0, B)</u>	—	(B, A)
<u>get(1)</u>	A	(B, A)
<u>set(2, C)</u>	<i>error</i>	(B, A)
<u>add(2, C)</u>	—	(B, A, C)
add(4, D)	<i>error</i>	(B, A, C)
remove(1)	A	(B, C)
add(1, D)	—	(B, D, C)
add(1, E)	—	(B, E, D, C)
get(4)	<i>error</i>	(B, E, D, C)
add(4, F)	—	(B, E, D, C, F)
set(2, G)	D	(B, E, G, C, F)
get(2)	G	(B, E, G, C, F)

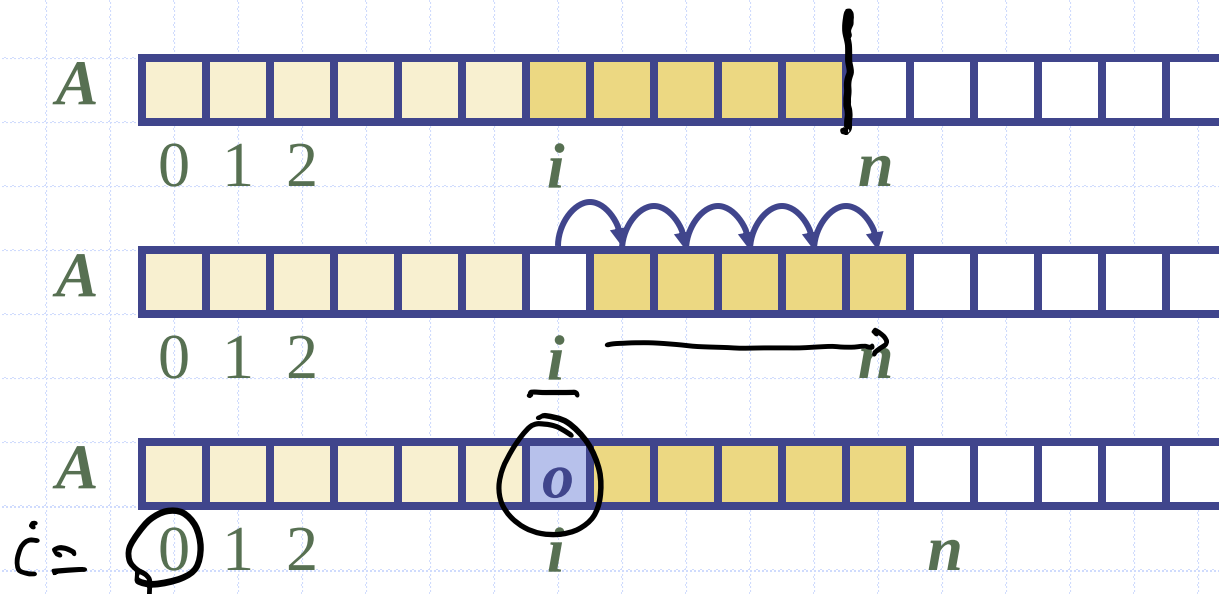
◦ Array Lists = Extensible Lists =
Growable Arrays = Python Lists = ...

- An obvious choice for implementing the list ADT is to use an array, A
- get(i) and set(i, e)



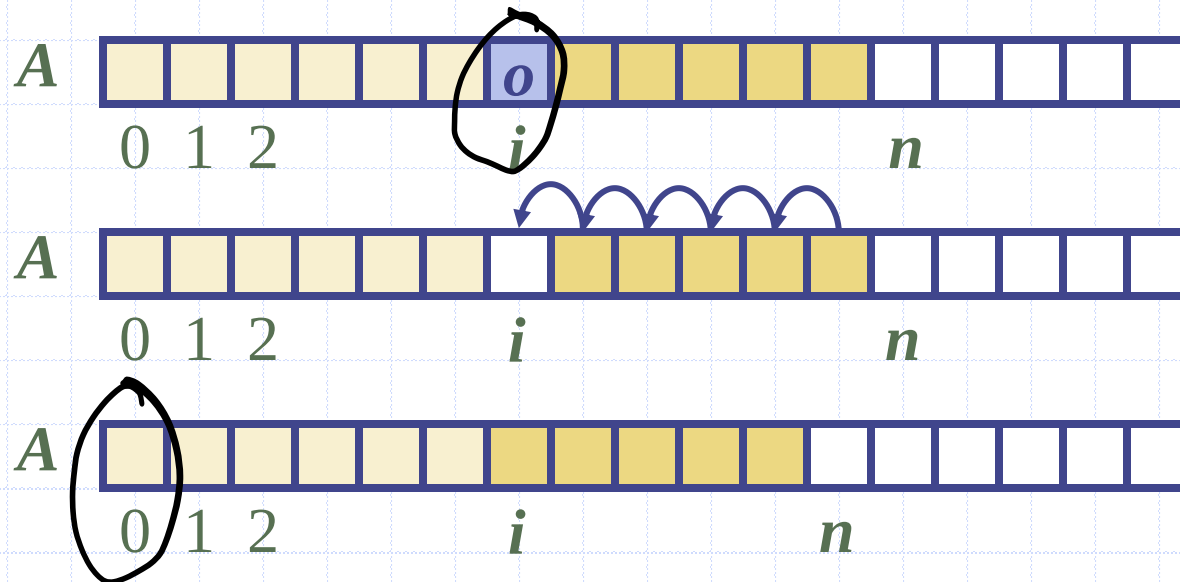
Insertion

- $add(i, o)$ – make room for the new element
- Worst case ($i = 0$), takes $O(n)$ time



Removal

- ❑ **remove**(i) – fill the hole left by the removed element
- ❑ Worst case ($i = 0$), takes $O(n)$ time



Performance

- Array-based implementation of a dynamic list
 - space used by the data structure is $O(n)$
 - accessing the element at i takes $O(1)$ time
 - add and remove run in $O(n)$ time
- add – when array is full
 - instead of throwing an exception (static Seq.) *
 - replace the array with a larger one ...

Extensible List

- **push(o)/add(o)/append(o)**
adds element **o** at the end
of the list

}

- How large should the new array be if we run out of capacity?

- **Incremental strategy**

- increase size by a constant c

- **Doubling strategy**

- double the size

|

|

Extensible List

- **push(o)/add(o)/append(o)**
adds element **o** at the end
of the list

Algorithm *push(o)*

if $\text{capacity} = S.\text{length} - 1$ **then**

$A \leftarrow$ **new array of size** *[something larger]*

$\text{capacity} =$ *[new larger value]*

for $i \leftarrow 0$ **to** $n-1$ **do**

$A[i] \leftarrow S[i]$ *// copy stuff!*

$S \leftarrow A$ *// update reference to new list*

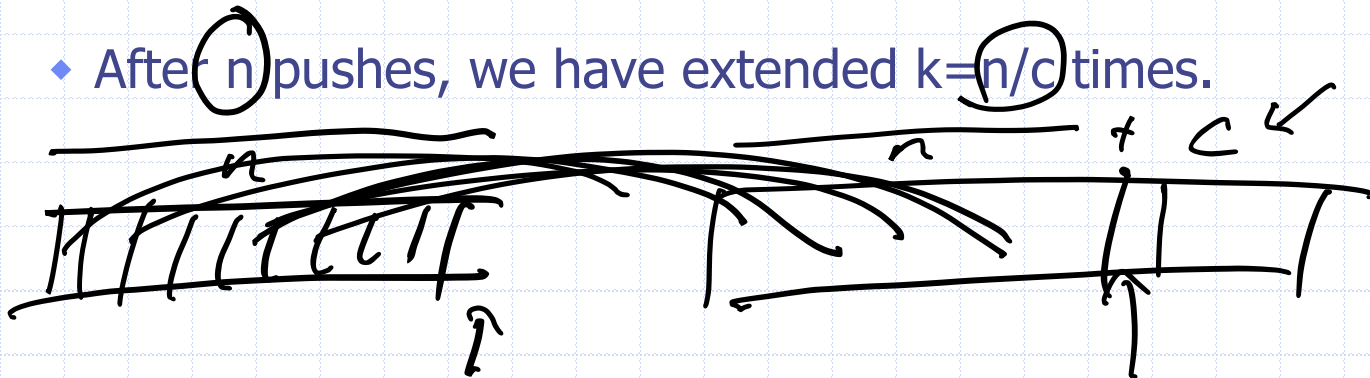
$S[n] \leftarrow o$

Comparison of Strategies

- Compare incremental and doubling strategies
 - Analysing total time $T(n)$ needed to perform a series of n push operations
- Amortised time of a push operation is the average time taken by a push operation over the series of operations
 - i.e. $T(n) \div n$
 - *Amortization Intuition: Chocolate...*

Incremental Strategy Analysis

- Over n push operations, array is replaced $k = n/c$ times, where c is a constant
 - EG: If we extend by $c=4$ elements each time:
 - ♦ After 4 pushes, we extend ($n = 4$, $k = 4/4 = 1$)
 - ♦ After 4 more pushes, we extend again ($n = 8$, $k = 8/4 = 2$)
 - ♦ ...
 - ♦ After n pushes, we have extended $k = n/c$ times.



Incremental Strategy Analysis

- Over n push operations, array is replaced $k = n/c$ times, where c is a constant
- Total time $T(n)$ of a series of n push operations is proportional to

$$\begin{aligned} & \underline{n} + \underline{c} + \underline{2c} + 3c + 4c + \dots + kc = \\ & \quad \underline{n + c(1 + 2 + 3 + \dots + k)} = \\ & \quad \underline{n + c(k(k+1)/2)} \end{aligned}$$

- Since c is a constant, $T(n)$ is $O(n + k^2)$, i.e. $O(n^2)$
- Thus, the amortised time of push is

$$\underline{T(n) / n = n^2/n = O(n)}$$

Doubling Strategy Analysis

- Array is replaced $k = \log_2 n$ times
- Total time $T(n)$ of a series of n push operations is proportional to

$$\underbrace{n}_{\text{push}} + \underbrace{1}_{\text{push}} + \underbrace{2}_{\text{push}} + \underbrace{4}_{\text{push}} + \underbrace{8}_{\text{push}} + \dots + \underbrace{2^k}_{\text{push}}$$

$$= n + 2^{k+1} - 1 \text{ [geometric series]}$$

$$= n + 2(2^k) - 1$$

$$= n + 2 \left(\underbrace{2^{\log_2 n}}_n \right) - 1$$

$$= \underline{3n - 1}$$

$$n + 2n - 1$$

$$\underbrace{2^{\log_2 n}}_n$$

$$\frac{O(n)}{n} \Rightarrow O(1)$$

- $T(n)$ is $O(n)$
- Amortised time of push is $T(n) \div n = O(1)$

“Bonus” Material

- Is Joel out of time AGAIN??? Yes
- Positional Lists
 - Interesting idea, but not critical to our learning
 - We can investigate if time permits
 - Otherwise, feel free to study this on your own

Positional Lists

- Our previous data structures all assume that update operations will happen at the ends of the structure
- What if we need to model more complex situations where insertions or deletions occur at arbitrary positions?
- **Observation:** If we had *some* node of a linked list, we can easily insert something ahead of it (or before it, if it's a doubly linked list)
 - So, design a structure that can use this observation...

Positional Lists

- A position acts as a marker
- We use positions when accessing/modifying data
- A position p is unaffected by changes elsewhere in a list
- A position instance is a simple object which returns the underlying data at position p .
 - `P.getElement()`

Positional List ADT: Access

`first()`

`last()`

`before( $p$ )`

`after( $p$ )`

`isEmpty()`

`size()`

Positional List ADT: Updates

`addFirst( $e$ )`

`addLast( $e$ )`

`addBefore( $p, e$ )`

`addAfter( $p, e$ )`

`set( $p, e$ )`

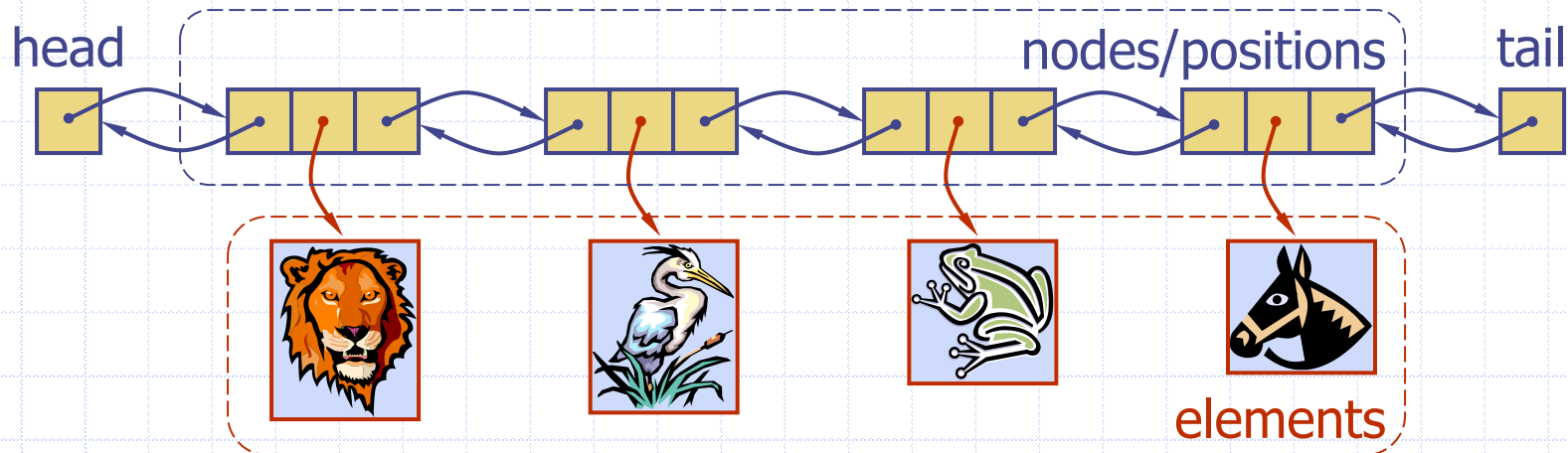
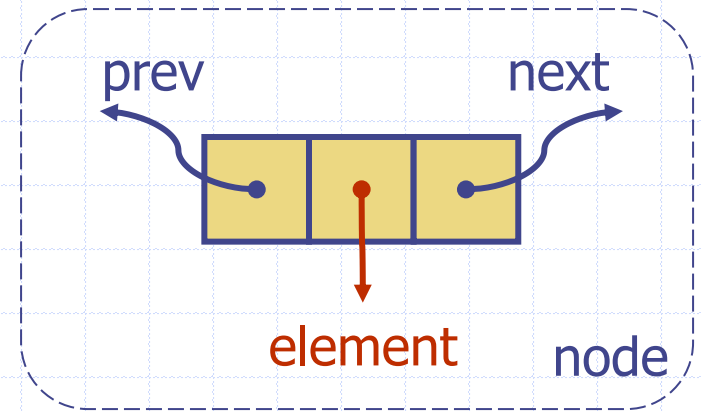
`remove( $p$ )`

Example

□ Sequence of Positional List operations

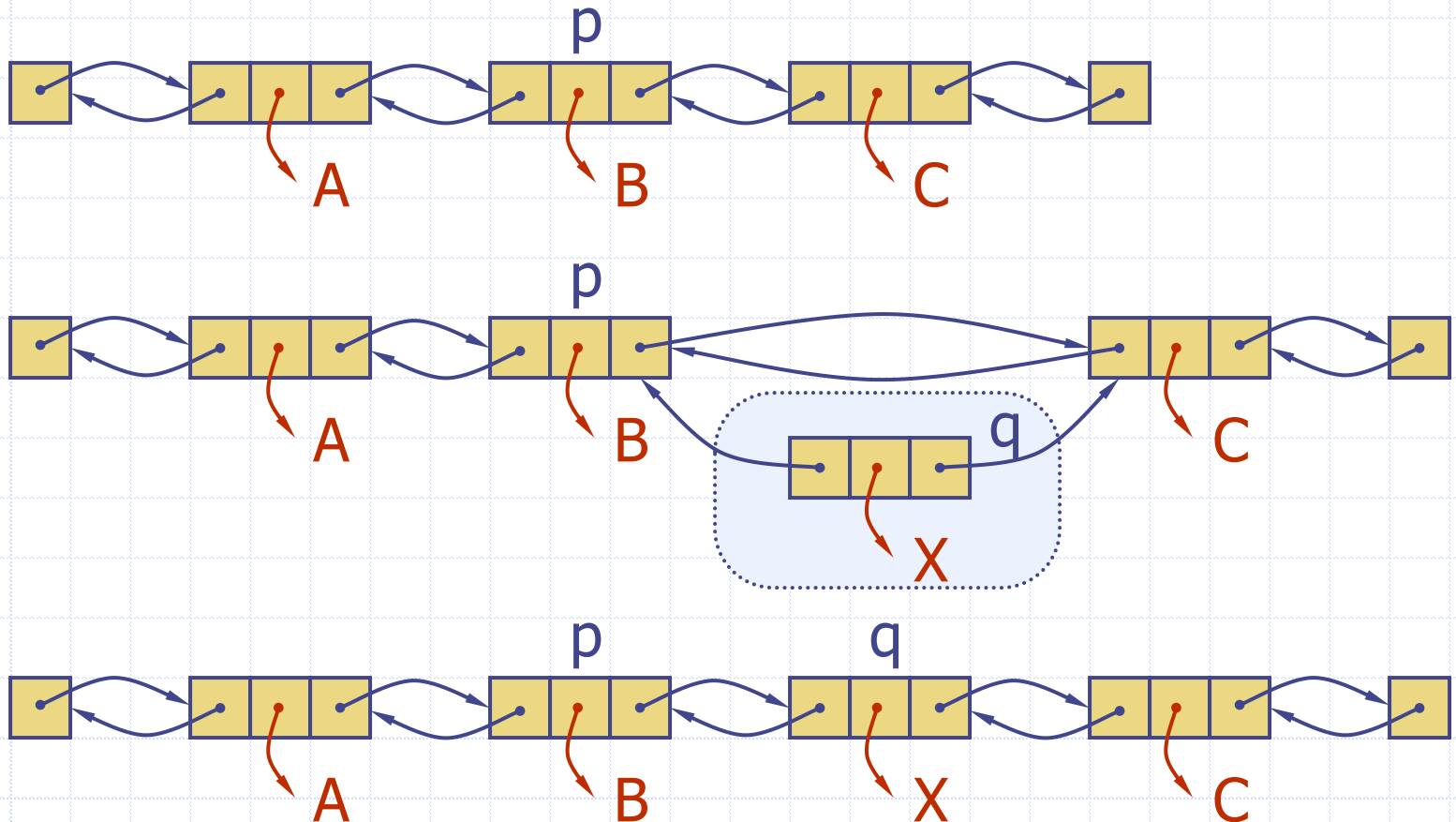
Method	Return Value	List Contents
addLast(8)	<i>p</i>	(8 <i>p</i>)
first()	<i>p</i>	(8 <i>p</i>)
addAfter(<i>p</i> , 5)	<i>q</i>	(8 <i>p</i> , 5 <i>q</i>)
before(<i>q</i>)	<i>p</i>	(8 <i>p</i> , 5 <i>q</i>)
addBefore(<i>q</i> , 3)	<i>r</i>	(8 <i>p</i> , 3 <i>r</i> , 5 <i>q</i>)
<i>r</i> .getElement()	3	(8 <i>p</i> , 3 <i>r</i> , 5 <i>q</i>)
after(<i>p</i>)	<i>r</i>	(8 <i>p</i> , 3 <i>r</i> , 5 <i>q</i>)
before(<i>p</i>)	null	(8 <i>p</i> , 3 <i>r</i> , 5 <i>q</i>)
addFirst(9)	<i>s</i>	(9 <i>s</i> , 8 <i>p</i> , 3 <i>r</i> , 5 <i>q</i>)
remove(last())	5	(9 <i>s</i> , 8 <i>p</i> , 3 <i>r</i>)
set(<i>p</i> , 7)	8	(9 <i>s</i> , 7 <i>p</i> , 3 <i>r</i>)
remove(<i>q</i>)	error	(9 <i>s</i> , 7 <i>p</i> , 3 <i>r</i>)

Positional List Implementation



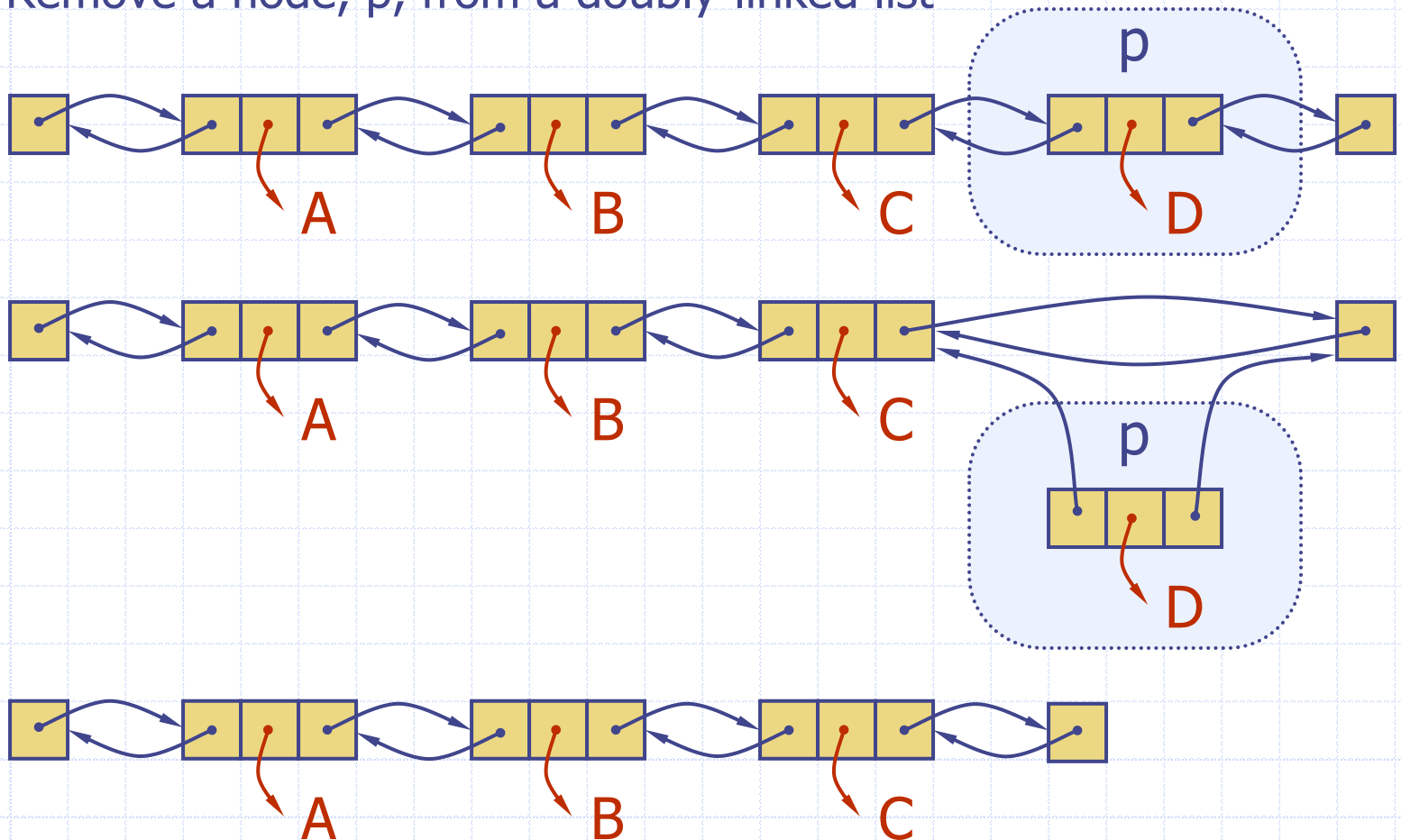
Insertion

- Insert a new node, q , between p and its successor



Deletion

- Remove a node, p , from a doubly-linked list



Further Reading

- ❑ Data Structures and Algorithms in Python
 - Chapter 5
 - Chapter 7.1 to 7.3
 - Chapter 12.4
- ❑ Introduction to Algorithms
 - Chapter 8
 - Chapter 10.1, 10.2